

Securing Nested Attestation of Confidential Serverless Computing without Intra-Enclave Isolation

Atsuki Momose

Acompany Co., Ltd.

atsuki.momose@acompany-ac.com

Ao Sakurai

Acompany Co., Ltd.

ao.sakurai@acompany-ac.com

Kailun Qin

Intel & Shanghai Jiao Tong University

kailun.qin@intel.com

Mona Vij

Intel Labs

mona.vij@intel.com

ABSTRACT

Confidential Computing-as-a-Service has gained significant attention in recent years, driven by rapid advances in Trusted Execution Environment (TEE) technology. Among various architectures, confidential serverless computing has emerged as a promising model. A common approach to designing confidential serverless computing involves decoupling the client workload from the initial enclave image and dynamically provisioning the workload at runtime. This enables both offloading the costly enclave bootstrapping and maintaining a fixed reference measurement for attestation. This decoupling necessitates *nested attestation*, where the client’s workload is attested via a custom attestation module embedded in a platform-attested enclave established at boot time. The challenge in designing nested attestation, however, is to distinguish fresh enclaves from the used ones to prevent *enclave reuse*. Specifically, a previously used enclave may be compromised and its attestation module tampered with. If the enclave is reused for another workload, it could bypass runtime attestation, allowing unverified code to execute.

In this paper, we present a novel approach to securing nested attestation for confidential serverless computing on Intel® Software Guard Extensions (Intel® SGX). Unlike prior works, our approach does not rely on intra-enclave isolation techniques to sandbox the client’s workload. Instead, we leverage the Key Separation and Sharing (KSS) feature of Intel SGX to track and prevent enclave reuse based on its immutable IDs. We develop a prototype system for confidential serverless computing for Python workloads, incorporating our nested attestation scheme, and present an empirical evaluation of its performance.

We believe our scheme unveils a unique yet previously overlooked role of KSS—*post-compromise identification*, with broader implications beyond serverless computing. As a concrete example, we demonstrate how KSS can be leveraged to implement an enclave-binding TPM on Intel SGX, which is of independent interest.

CCS CONCEPTS

• Security and privacy → Distributed systems security;

KEYWORDS

Confidential Computing; Attestation; Intel SGX

1 INTRODUCTION

Confidential Computing-as-a-Service (CCaaS) [7, 44] is a form of cloud service model where the cloud service provider (CSP) offers *secure enclaves*, typically backed by hardware-assisted Trusted Execution Environment (TEE) technology, allowing users to run workloads in an isolated execution environment that remains protected from system software, hypervisors, and even cloud administrators. Common TEE technologies include Intel SGX [11, 28] and Intel TDX [17], AMD SEV [3], and ARM CCA [24]. These technologies typically provide tamper-proof attestation features, allowing remote clients to verify both the hardware specifications and the exact software running within the enclave. This ensures remote clients can securely execute their workloads even against a potentially malicious CSP.

Among various cloud service models, serverless computing is a promising architecture for confidential computing, as it enables the on-demand execution of minimal, well-defined Trusted Computing Bases (TCBs) within an application. Among the available TEE technologies, Intel SGX is particularly well-suited for building confidential serverless computing, as it is the only solution that provides fine-grained memory isolation and secure execution of small, ephemeral workloads [2, 22, 32, 44], and has seen wide adoption in production [10, 30].

A common design technique in general (non-confidential) serverless computing backends involves preparing an *instance template*—a container pre-initialized with platform components such as the language runtime—while keeping the serverless workload itself as a placeholder prior to receiving a request [20, 23, 25]. Upon request, the designated workload is loaded into the prepared instance template and executed immediately. This allows the platform to offload the costly process of full instance creation and respond to requests with minimal latency.

This technique becomes even more crucial in confidential serverless computing from both performance and usability perspectives. From a performance standpoint, instance creation involves enclave bootstrapping, which requires computationally intensive cryptographic operations, such as computing a hash measurement of the entire enclave memory. Thus, offloading instance bootstrapping significantly reduces the client’s online latency in confidential serverless computing [44].

From a usability perspective, decoupling the serverless workload from the initial enclave image is essential for client attestation. If the workload is embedded in the enclave at boot time, the hardware measurement will vary for each deployment. As a result, the client

must recalculate the expected measurement separately for every new workload. This not only imposes an unnecessary computational burden but also forces the client to retain the entire enclave image including platform components—despite much of it being irrelevant to the workload. With a single base image for the boot-time enclave and the serverless workload decoupled, the client can reuse a fixed reference measurement for attestation, which can be precomputed once locally or even authenticated by trusted third parties.

With the workload decoupled from the boot-time enclave, a *nested attestation* mechanism [44] is crucial to assure clients that their requested workload executes with integrity. Specifically, the client first obtains and verifies a hardware-signed attestation report (or *quote*) for the instance template, validating it against a trusted reference measurement. Once the template is attested, a custom attestation module embedded in the template generates and signs an attestation report for the workload—such as its hash measurement—which the client receives and verifies to ensure the workload’s integrity before execution.

However, designing a secure nested attestation protocol is not as straightforward as one might expect. The challenge is to reliably distinguish fresh enclaves from used ones to defend against *enclave reuse*. Specifically, suppose a corrupted workload is executed on an instance, and the enclave, including the custom attestation module, is tampered with. If a malicious CSP reuse the compromised enclave for another client’s workloads, the client remains unaware of this runtime tampering. This is because the hardware-signed attestation report reflects only the measurement of the boot-time enclave, rather than its latest state. Consequently, the malicious CSP could execute an unverified arbitrary workload on the victim client’s runtime data, resulting in a breach of confidentiality and integrity.

One common solution to mitigating this threat is to adopt *intra-enclave isolation* to sandbox workload executions and protect the enclave from potentially corrupted workloads [1, 2, 15, 35, 37, 44]. The attestation module in the protected enclave region could block any subsequent execution to prevent enclave reuse or sanitize the memory to enable secure reuse [44]. However, Intel SGX does not provide built-in support for intra-enclave isolation, such as privilege separation. Therefore, most previous works have adopted compiler-based Software Fault Isolation (SFI) techniques [1, 35, 37, 44]. The main drawback of SFI, however, is its limited software compatibility. Because SFI requires recompiling source code to insert memory access control mechanisms—such as Control-Flow Integrity (CFI) instructions—it is incompatible with off-the-shelf binaries. Additionally, CFI mechanisms introduce non-negligible runtime overhead. From a security perspective, SFI increases the TCB size due to the additional enforcement logic, including CFI instrumentation and a verification module for SFI policy compliance. This added complexity may expand the attack surface and introduce new vulnerabilities.

Our Result. In this paper, we present a novel approach to securing nested attestation for confidential serverless computing on Intel SGX. Our scheme does not rely on any intra-enclave isolation techniques and thus provides binary-level compatibility for workloads. Our key idea is to leverage the Key Separation and Sharing (KSS) feature of Intel SGX to track and prevent enclave reuse. Specifically,

each *worker* enclave, which executes a client workload, is managed by an *attester* enclave. The attester tracks *fresh* workers and ensures that workloads are never assigned to previously used workers, which may have been compromised by prior executions. The core challenge is to prevent a compromised worker from re-attesting to an attester in an attempt to get reassigned a new workload. To address this challenge, we utilize `CONFIG_ID`—one of the extended IDs provided by KSS—to bind an attester-generated random worker ID to the worker. Since `CONFIG_ID` is immutable after enclave initialization and included in the hardware-signed attestation report, the attester can reliably distinguish fresh workers from reused ones, even after compromise. We also utilize `CONFIG_ID` to achieve mutual attestation between an attester and a worker [6]. In particular, it helps resolve cyclic dependencies in their measurements, while preserving attestation security even in the presence of a compromised worker.

We believe our use case for KSS is quite unique and is of independent interest. To elaborate, KSS—as the name suggests—was initially introduced for fine-grained control of key derivation based on extended enclave identities. However, our scheme unveils a unique role of KSS that has been somewhat overlooked—*post-compromise identification*. While traditional Intel SGX mechanisms such as `MR_ENCLAVE` and `MR_SIGNER` primarily focus on ensuring an enclave’s initial integrity, they do not help distinguish enclaves that have been compromised after launch. In contrast, KSS extended IDs remain immutable throughout the enclave’s lifetime, allowing reliable distinction between enclaves even post-compromise. This is particularly powerful in the emerging field of Confidential Computing-as-a-Service (CCaaS) where programs beyond initial TCBS are dynamically provisioned and potentially corrupted.

To showcase the broader implications of our KSS use case beyond serverless computing, we also demonstrate how it can be utilized to implement an *enclave-binding* Trusted Platform Module (TPM)—i.e., an enclave-local trusted functionality virtually integrated into each enclave. In particular, our approach realizes an Intel SGX counterpart to Intel TDX’s Runtime Measurement Registers (RTMRs) [9], which support runtime measurement of dynamic enclave state via hash extensions, similar to TPM PCRs [33]. This immediately expands the functional capabilities of Intel SGX, and its implementation is a promising future work.

We have developed a prototype system for confidential serverless computing built on top of our nested attestation scheme. Our prototype follows a Function-as-a-Service (FaaS)-like architecture and is built on the open-source Library OS, Gramine-SGX [40]. It supports workloads implemented as standard Python packages. While the entry point (or *handler*) for workload execution is limited to Python, it can invoke any Linux binaries, such as shared libraries, as commonly done in standard Python packages. Our prototype also supports multi-user workloads (i.e., supporting multiple sources of runtime inputs) and can be used in Multi-Party Computation (MPC) settings, a common real-world use case for confidential computing.

We also present an empirical evaluation of our prototype using machine learning (ML) training and prediction workloads involving two clients engaged in collaborative data analysis—a common industry use case for confidential serverless computing. We provide a detailed latency breakdown of each step in our protocol and

identify the primary sources of overhead. In particular, we confirm the substantial benefit of offloading enclave bootstrap phase, which introduces significant overhead that scales linearly with the enclave size. Additionally, we observe that dynamic workload provisioning and its associated integrity verification process become a major bottleneck when building confidential serverless systems on Gramine, and we propose a potential feature extension to Gramine to resolve this bottleneck.

Summary of Contributions.

- We present a nested attestation scheme for confidential serverless computing on Intel SGX without relying on intra-enclave isolation. We introduce a novel technique that prevents enclave reuse by leveraging KSS (Section 4).
- We develop a prototype system of confidential serverless computing that incorporates our nested attestation protocol and presents an empirical evaluation (Section 5).
- To demonstrate the broader implication of our use case of KSS beyond serverless computing, we show how to implement an enclave-binding TPM on Intel SGX (Section 6).

Organization. The remainder of this paper is organized as follows. Section 2 reviews technical background and clarifies the system model and problem definition. Section 3 outlines the key techniques in our nested attestation scheme. Section 4 presents the detailed design of our nested attestation protocol. Section 5 describes our prototype implementation of confidential serverless computing and provides an empirical evaluation. Section 7 discusses related work, and Section 8 concludes the paper.

2 BACKGROUND

The primary goal of this work is to secure nested attestation in confidential serverless computing on Intel SGX without relying on intra-enclave isolation. In this section, we clarify the system model, define our adversary assumptions, and formalize the problem of confidential serverless computing and nested attestation.

2.1 Confidential Serverless Computing

Confidential Computing-as-a-Service (CCaaS) is an emerging paradigm of cloud computing that enables secure execution of workloads within TEE, ensuring that data remains protected even from cloud service providers. Among various cloud service models, *serverless computing* is particularly promising for confidential computing.

Serverless computing is a cloud service model that abstracts infrastructure management and eliminates the need for long-lived, privileged system software, allowing on-demand execution of essential workloads.

Function-as-a-Service (FaaS) is a widely adopted form of serverless computing, where custom stateless logic, called a *function*, is deployed by an application developer and executed on demand, enabling applications to seamlessly scale up and down. Specific FaaS services include AWS Lambda, Google Cloud Functions, and Azure Functions.

The on-demand execution of small, ephemeral workloads makes serverless computing particularly well-suited for confidential computing, as it enables the isolation of a minimal TCB, which aligns well with the design philosophy of traditional trusted computing.

2.2 Intel SGX

Among various available TEE technologies, Intel® Software Guard Extensions (Intel® SGX) is particularly well-suited for confidential serverless computing. Intel SGX is a set of micro-architectural extensions that enables hardware-backed isolation of user processes. It creates an isolated execution environment called an *enclave*, protected within a dedicated memory region called the Enclave Page Cache (EPC), which stores enclave data in an encrypted form, ensuring that code and data remain hidden from all privileged software including OS and hypervisor.

Remote Attestation. Intel SGX offers a remote attestation feature, allowing remote clients to verify the enclave’s integrity and authenticity. Specifically, each enclave can request an attestation report, called a *quote*, from the platform to prove its integrity and authenticity to a remote user. The quote is signed using a platform-derived key, which is authenticated through Intel’s public-key infrastructure (PKI).

The quote body contains specifications of both the platform and the enclave. Concretely, it contains the measurement of the enclave, `MR_ENCLAVE`, the cryptographic hash of the initial memory map at launch, allowing users to verify the initial integrity of the enclave. The quote body also includes `REPORT_DATA`, a 64-byte field that can hold arbitrary enclave-defined data, allowing the enclave to bind its runtime data, most often a hash of a signature verification key.

Notably, neither of the above fields is reliable post-compromise. Specifically, `MR_ENCLAVE` only reflects the initial enclave state and is never updated after launch. Therefore, if the enclave is compromised later, it is undetectable to clients. Similarly, `REPORT_DATA` is set within the enclave. Thus, a compromised enclave could fabricate arbitrary runtime data.

Key Separation and Sharing (KSS). KSS is an extension to the Intel SGX enclave identification scheme that introduces a suite of additional identifiers for each enclave instance, enabling finer-grained identification beyond `MR_ENCLAVE`.

- (1) `ISV_FAMILY_ID` and `ISV_EXT_PROD_ID` (16 bytes each) are specified by the Independent Software Vendor (ISV) during enclave development and are included in the enclave definition (i.e., the enclave signature structure).
- (2) `CONFIG_ID` (64 bytes) and `CONFIG_SVN` (2 bytes) are specified during enclave initialization and are therefore suitable for runtime identification.

As the name suggests, KSS was originally introduced for fine-grained control of key derivation based on these extended IDs. However, we observe that KSS is particularly useful for *post-compromise identification* of enclaves. To elaborate, all of these extended IDs are stored in the Intel SGX Enclave Control Structure (SECS). During runtime, SECS is stored in a dedicated memory range within the EPC and is protected from both the untrusted host system and the enclave itself. Thus, KSS extended IDs remain immutable

throughout the enclave’s lifetime, even post-compromise. Additionally, these extended IDs are all included in the quote body and can be verified by remote clients.

2.3 Adversary Model

To clarify the trust model, let us define the adversary and its assumption here. We assume a strong adversary who has full control over the CSP, including all system software components, including OS, hypervisor. The adversary could be malicious and behaves arbitrarily, such as inspecting, deleting, or modifying memory, I/O operations, or network communications. However, we assume that Intel SGX is secure, meaning the adversary cannot access or modify enclave memory beyond what is explicitly allowed by the enclave program through defined interfaces.

While advanced microarchitectural side-channel attacks [4, 16, 41] on Intel SGX have been studied, we consider them out of scope for this work. We also note that application-level side channels, such as enclave response latency or structural patterns in encrypted data, may leak sensitive information. While such threats can be mitigated through techniques such as constant-time programming [5, 42] or the use of oblivious data structures [14, 38, 39], we consider them to be the responsibility of workload implementation.

Cryptographic Assumptions. We assume that the adversary is computationally bounded and cannot break any cryptographic primitives. This aligns with the security assumptions of the underlying confidential computing and TEE technologies. When we state that a value (e.g., random value or ID) is of sufficient length, we mean that it is on the order of the computational security parameter.

We assume the use of digital signatures, where any signature is unforgeable without knowledge of the corresponding private key. Additionally, we assume the use of message authentication codes (MACs) for ensuring data integrity and cryptographic hash functions for collision resistance.

We assume the use of standard symmetric key encryption (such as AES) for securing data transmission between nodes (i.e., system components) that share a secret key. The shared secret key is commonly established using the standard Diffie-Hellman (DH) key exchange, once the nodes certify their respective signature public keys, as in standard TLS handshakes. In our context (and confidential computing in general), public keys are certified as part of the attestation process [18], depending on the specific attestation scheme.

2.4 Basic System Architecture

The typical serverless computing model consists of three main components. A remote *client* (i.e., the cloud user) submits workload execution requests to the service hosted by CSP, which are handled by a frontend *gateway controller* responsible for managing and routing requests. The actual execution is carried out by an ephemeral backend *worker*, which processes the assigned workload.

The goal of confidential serverless computing is to protect client’s workload execution from the adversary (i.e., malicious CSP). Thus, the worker operates inside an enclave, ensuring isolation from the untrusted host system. In summary, we consider confidential serverless computing consisting of the following basic components (also see Figure 1).

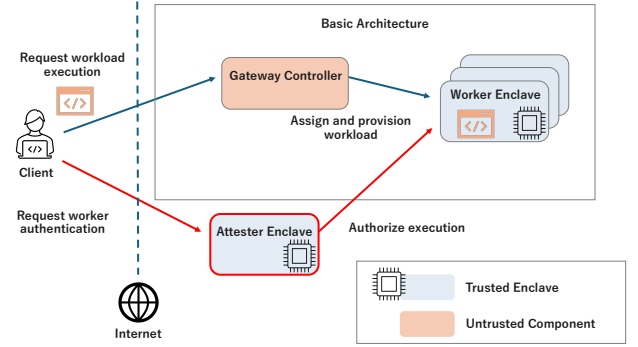


Figure 1: The system architecture for confidential serverless computing, with our new components and workflows highlighted in red.

- (1) **Client.** A client requests workload execution after verifying the integrity and authenticity of the worker enclave and establishing a secure channel to transmit sensitive runtime data. Multiple clients may be involved in submitting runtime inputs to the same workload—i.e., in a Multi-Party Computation (MPC) setting. In that case, the workload logic is pre-agreed and may be deployed by one of the clients.
- (2) **Worker Enclave.** A dedicated enclave that executes the assigned workload. Each workload is executed in a newly created worker enclave, meaning multiple worker enclaves may exist concurrently. The initial enclave image of a worker includes *trusted* components such as the language runtime, essential dependencies, and system-specific middleware for workload execution as well as external interaction. The *untrusted* workload is provisioned at runtime and executed on top of this middleware.
- (3) **Gateway Controller.** The gateway controller is responsible for managing worker enclaves and handling workload requests. It is an *untrusted* component fully controlled by the adversary. Concrete examples of gateway controllers include an ingress proxy, a Kubernetes orchestrator, and other programs running outside the enclave.

Remark on Client-CSP Collusion. In the multi-client setting, we assume that some clients may collude with the CSP to breach the confidentiality or integrity of the workload. More concretely, we adopt the standard dishonest-majority MPC model under a malicious adversary [12, 13, 21], where all corrupted parties (including clients and CSP) may collude to compromise the security of honest clients. However, as is commonly assumed in MPC literature, we consider input authenticity (i.e., whether a client submits a valid input) to be out of scope.

Remark on Persistent State. Throughout this paper, we do not consider any persistent (offline) state. Our attestation protocol, as well as the underlying serverless computing system, maintains no persistent data across sessions. All enclaves and the gateway controller erase their state upon shutdown and are re-initialized from a clean state upon restart. We note that the absence of persistent state

is particularly desirable in the context of confidential computing, as it eliminates the need for a *sealing* capability and mitigates related threats such as replay attacks [26].

2.5 Problem Definition: Nested Attestation

Our ultimate goal is to achieve confidential serverless computing that allows a client to deploy and execute a workload while ensuring that its runtime data remains protected from an adversary (i.e., malicious CSP). More formally, given a client-defined workload F and a runtime input x , the client aims to compute $F(x)$ on the service while preventing any unauthorized access to x or intermediate computations—a.k.a., *secure remote computation*.

In the design of confidential serverless computing, it is generally preferable to decouple the client workload F from the worker’s initial enclave image and dynamically provision it at runtime. As elaborated in Section 1, this enables offloading the enclave bootstrapping overhead and maintaining a fixed reference measurement for attestation usability. However, this poses an additional challenge in ensuring the integrity and authenticity of the workload. While the boot-time enclave can be verified using the hardware-signed measurement, the runtime-provisioned workload lacks built-in integrity guarantees and must be attested separately via a trusted attestation module embedded in the verified boot-time enclave—hence the name *nested attestation*.

The primary challenge in designing a secure nested attestation scheme is to defend against *enclave reuse*. Specifically, an adversary (i.e., malicious gateway controller) could attempt to reuse a previously compromised enclave to execute a new workload, bypassing runtime attestation. Consider the following straightforward attack: A malicious gateway controller, acting as a dummy client, provisions a malicious workload into a worker, which compromises the enclave memory, including the attestation module. The gateway controller then reuses this compromised worker to execute an unverified arbitrary workload on a victim client’s runtime data. Since this runtime tampering is undetectable from the client, the client will send the sensitive runtime data to the compromised worker, resulting in a confidentiality and integrity breach. Beyond such dummy executions, enclave compromise may also arise due to vulnerabilities in the workload itself or from accepting dynamic scripts as input, potentially enabling arbitrary code execution. Thus, our goal is to defend against enclave reuse by a malicious gateway controller, regardless of the underlying security assumptions about the workload.

One common approach to protect against enclave reuse is to apply intra-enclave isolation [37, 44], such as Software Fault Isolation (SFI), for sandboxing the workload. Specifically, SFI enforces fine-grained memory access control within an enclave by instrumenting the workload binary with Control-Flow Integrity (CFI) checks and restricting its execution environment. This prevents a malicious workload from compromising the enclave’s memory outside its designated region. However, as elaborated in Section 1, SFI requires recompilation, limiting software compatibility and increasing runtime overhead as well as the TCB due to additional enforcement mechanisms, which we therefore avoid in this work.

3 OVERVIEW

This section provides an overview of our nested attestation scheme. As discussed in Section 2.5, the primary security threat in nested attestation is *enclave reuse*, where a compromised enclave could be reassigned to execute a new workload.

System overview. Our basic idea is to track and control worker enclaves from a centralized, well-defined TCB called an *attester* enclave (Figure 1). The attester enclave maintains exclusive control over workers via an authenticated channel, established through mutual attestation along with a Diffie-Hellman (DH) key exchange. All instructions from the attester to a worker are MAC-signed using the shared secret key and are verified by the worker before execution. The attester authorizes workload execution on each worker through MAC-signed instruction. Once a workload execution is authorized, the corresponding worker is marked as *contaminated* within the attester and is never authorized for any future execution. To securely realize this mechanism, we leverage two techniques based on KSS, as elaborated below.

3.1 Enclave Tracking using Config ID under Potential Enclave Compromise

Up until the workload execution, the worker’s behavior is well-defined and entirely under the attester’s control. However, once execution begins, the worker enclave becomes completely untrusted. Since we do not rely on any intra-enclave isolation, a compromised workload could take full control of the enclave, rendering any message sent from the worker untrusted. As a result, a compromised worker could attempt to re-connect to an attester (either original or perhaps a newly created attester by a malicious CSP), impersonating a newly created worker to get reassigned a workload.

Our key technique to defend against this threat is to bind an attester-managed random worker ID to the `CONFIG_ID`, which serves as a post-compromise identifier. Specifically, when a worker enclave is initialized, its `CONFIG_ID` is set to a random worker ID generated by an attester. The worker ID is managed locally by the attester in its *fresh worker pool* until the workload execution on the worker is authorized. During the mutual attestation process, the attester verifies that the `CONFIG_ID` in the worker’s quote is contained in its fresh worker pool. If so, the attester treats the worker as newly created and establishes an authenticated channel. This ensures a strong, exclusive binding between the attester and the worker. If the worker attempts to reconnect to a newly created attester, the worker ID bound to it is unlikely to appear in the attester’s fresh pool—since worker IDs are randomly generated and collisions across different attesters are rare. Similarly, if the worker attempts to reconnect to the original attester, its worker ID should no longer be present in the fresh pool, as the workload execution must have already been authorized by this attester. In either case, the worker’s reconnection attempt should be detected and blocked by the attester.

After the worker comes under the attester’s exclusive control, the client verifies the worker’s authenticity indirectly via the attester. Specifically, the client verifies the attester’s quote, which includes metadata about the assigned worker, bound to `REPORT_DATA`. The

metadata includes the worker’s public key and the assigned workload, as requested by the client. The authenticated worker’s public key is then used to establish a shared secret with the worker, enabling secure transmission of sensitive data required for execution.

It is worth noting that `REPORT_DATA` is unsuitable for binding worker IDs. As elaborated in Section 2.2, while `REPORT_DATA` is commonly used to prove runtime data from within an enclave, it is only reliable if the enclave is trusted. Similarly, ISV IDs (i.e., `ISV_FAMILY_ID` and `ISV_EXT_PROD_ID`) do not meet our requirements, as they are defined at enclave build time (i.e., in the enclave’s signature structure) and cannot be assigned dynamically at runtime.

3.2 Mutual Attestation using Config ID under Potential Enclave Compromise

So far, we have assumed that the attester and worker mutually attest to each other, ensuring that they communicate exclusively—i.e., preventing attackers from impersonating either enclave. Specifically, the attester must verify the worker’s measurement to ensure that the authenticated worker metadata originates from a genuine worker enclave. Otherwise, a fabricated worker’s public key could be used by the client, potentially leaking the client’s runtime data. Similarly, the worker must verify the attester’s measurement to ensure that it is controlled exclusively by a legitimate attester—preventing attackers from initiating workload execution without proper authorization.

However, mutual attestation is not as straightforward as standard unidirectional attestation, especially under the threat of enclave compromise. The key challenge lies in resolving cyclic dependencies in the measurement process. More concretely, suppose the attester hardcodes the worker’s measurement within its enclave image. In that case, the worker must be built first, before the attester. However, this prevents the worker from embedding the attester’s measurement in return, since the attester’s measurement does not yet exist at the time of the worker’s creation.

A straightforward solution is to supply the attester’s measurement to the worker at runtime. The worker uses this value as a reference measurement to verify the attester’s quote. Furthermore, the worker transfers the attester’s measurement, binding it to the `REPORT_DATA` in its own quote. In turn, the attester verifies the `REPORT_DATA`, comparing it against its own measurement. This confirms that the worker enclave has attested to the genuine attester. However, this approach implicitly assumes that the worker remains trustworthy throughout its lifecycle—`REPORT_DATA` is unreliable after compromise.

To realize this mutual attestation securely under potential worker compromise, we use `CONFIG_ID`. Specifically, the attester’s measurement is bound to the worker enclave’s `CONFIG_ID`. The worker then verifies the attester’s quote using the reference value stored in its own `CONFIG_ID`. When verifying the worker’s quote, the attester checks that the worker’s `CONFIG_ID` matches its own measurement. This guarantees that the worker enclave can only attest to the genuine attester.

4 OUR NESTED ATTESTATION PROTOCOL

This section presents the details of our nested attestation protocol for confidential serverless computing.

Protocol Outline. Our protocol consists of two phases: a *bootstrap* phase, in which workers are created, and an *execution* phase, in which workload is executed.

The bootstrap phase is independent of the client workload and can be completed before receiving any request. As discussed, this enables us to offload the enclave bootstrapping overhead and maintain fixed reference measurements, improving the usability of client attestation. This phase also includes mutual attestation between the attester and the worker, during which the worker becomes exclusively bound to and controlled by the attester.

In the execution phase, the client, assisted by the attester, verifies the integrity and authenticity of the worker, establishes a secure channel, and transmits runtime inputs to execute the workload.

4.1 Bootstrap Phase

We describe the bootstrap phase in Figure 2. The protocol involves three system components: the gateway controller, attester enclave, and worker enclave. The client can stay offline during this setup. The protocol consists of two main steps: the creation of a worker enclave and mutual attestation between the worker and the attester. The gateway controller and the attester enclave are created upon system initialization.

Worker enclaves are created by the gateway controller on demand. As discussed in Section 3.1, the first key technique to prevent enclave reuse is to bind an attester-generated random worker ID to `CONFIG_ID` and track workers from the attester. The attester records the worker ID in its *fresh worker pool*—i.e., the local set of worker IDs. During enclave initialization, the gateway controller binds the worker ID to `CONFIG_ID` via hash extension. The assigned worker ID is passed to the worker enclave, which verifies its integrity against its own `CONFIG_ID`. The attester, during mutual attestation step, checks that the worker ID bound to the worker’s `CONFIG_ID` is registered in its fresh worker pool; otherwise, the worker has been used and is potentially contaminated.

We note that the worker ID must be randomly sampled from a sufficiently large space. If worker IDs are not sufficiently random and long, collisions may occur across multiple attesters. This could allow a worker enclave with the same ID to be attested more than once, leading to enclave reuse.

As discussed in Section 3.2, mutual attestation between the attester and the worker is essential to establish an exclusive control channel, and this step involves our second technique leveraging KSS. Specifically, while the attester verifies the worker using a hard-coded reference measurement, the worker must receive the attester’s measurement at runtime to resolve the cyclic dependency between their measurements. Crucially, the attester’s measurement must be provided as an immutable value, allowing the attester to confirm that the worker attested only to the genuine attester. We therefore use `CONFIG_ID` to embed the attester’s measurement in the worker as a binding and immutable value that persists throughout the enclave lifecycle—even after compromise.

4.2 Execution Phase

Next, we describe the execution phase. The protocol is summarized in Figure 3 and consists of three main steps: workload assignment,

The protocol involves three system components: the gateway controller, attester enclave, and worker enclave. Clients can be offline.

- **Worker Creation.** Worker creation is performed on demand, triggered by the gateway controller, and consists of the following steps:
 - (1) The attester generates a random worker ID and records it in the fresh worker pool (i.e., the set of active worker IDs).
 - (2) The gateway controller receives the worker ID and creates the worker enclave. During initialization, CONFIG_ID is set to the hash of: 1) the assigned worker ID, and 2) the attester's MR_ENCLAVE.
 - (3) Once initialized, the worker enclave receives the worker ID and the attester's MR_ENCLAVE from the gateway controller and verifies that its CONFIG_ID correctly matches the hash of these values.
- **Mutual Attestation.** Once worker creation is complete, the worker and the attester perform mutual attestation as follows:
 - (1) The worker generates its public-private key pair and sends the attester: 1) its quote, with the hash of the worker's public key attached to REPORT_DATA, 2) its public key, and 3) the worker ID.
 - (2) If the worker ID is contained in the fresh worker pool and this is the first time to perform mutual attestation, then the attester verifies the worker's quote by checking that:
 - MR_ENCLAVE matches the expected measurement of the worker enclave, hard-coded in the attester enclave.
 - REPORT_DATA matches the hash of the worker's public key.
 - CONFIG_ID matches the hash of the worker ID and the attester's own MR_ENCLAVE.
 - (3) The attester generates its public-private key pair and sends the worker: 1) its quote, with the hash of the attester's public key attached to REPORT_DATA, and 2) its public key.
 - (4) The worker verifies the attester's quote by checking that:
 - MR_ENCLAVE matches the expected measurement of the attester enclave provided by the gateway controller during worker creation.
 - REPORT_DATA matches the hash of the attester's public key.

The attester and the worker then perform a Diffie-Hellman (DH) key exchange using their respective public keys to establish a shared secret key.

Figure 2: Our nested attestation protocol: Bootstrap phase

The protocol involves four system components: the client, the gateway controller, the attester enclave, and the worker enclave.

- **Workload Assignment.** Upon receiving a request from the client along with the client's public key, the requested workload is assigned to an available worker chosen by the gateway controller, and the workload ID is generated as follows:
 - (1) The gateway controller sends the attester: 1) the workload definition, 2) the worker ID of the chosen worker, and 3) the client's public key.
 - (2) If no workload has yet been assigned to the worker ID, the attester generates a workload ID as the hash of: 1) the workload definition, 2) the client's public key, and 3) a random nonce. The attester then stores the workload ID along with the worker ID.
 - (3) The assigned worker enclave receives the workload definition, the client's public key, and the attester's nonce, and computes the workload ID similarly. The workload body is also provisioned and verified against the given workload definition.
 - (4) The client also receives the attester's nonce and computes the workload ID similarly.
- **Client Attestation.** The client verifies the authenticity of the worker and establish a shared key as follows:
 - (1) The client generates a random nonce and sends it to the attester (via the gateway controller).
 - (2) The attester generates its quote with REPORT_DATA set to the hash of: 1) the worker's public key, 2) the workload ID, and 3) the client's nonce.
 - (3) The client receives the quote and the worker's public key and verifies that:
 - MR_ENCLAVE matches the expected measurement of the attester enclave.
 - REPORT_DATA matches the hash of: 1) the worker's public key, 2) the workload ID, and 3) the client's nonce.
 - (4) The client then performs a DH key exchange with the worker using their respective public keys, to establish a shared secret key.
- **Workload Execution.** Once attestation is complete, workload execution proceeds as follows:
 - (1) The client encrypts the runtime input for the workload using the shared secret key and sends it to the worker.
 - (2) The attester generates a MAC over the workload ID using the shared key established with the worker (during the bootstrap phase) and sends it to the worker. The attester then deletes the worker ID along with all associated data (i.e., workload ID, worker's public key).
 - (3) The worker verifies the MAC-signed workload ID, decrypts the runtime input from the client, and starts executing the workload.

Figure 3: Our nested attestation protocol: Execution phase.

client attestation, and workload execution. We highlight some key points below.

Workload Definition. We begin by noting that our protocol is agnostic to the specifics of the workload—such as its code structure, language runtime, or available kernel capabilities. To abstract away these details, we refer to them collectively as the *workload definition*. This may include the computing logic, required service data, or specific runtime configurations of the worker enclave. Our protocol

only assumes that the integrity of the workload (its body) can be verified against this definition. For a concrete example of a workload definition, please refer to Section 5.2, where we describe our prototype system.

Workload ID. The primary role of the workload assignment step is to generate a random *workload ID*. This ID ensures that each worker enclave is used only once, regardless of the workload definition. More specifically, a malicious gateway controller may attempt to

reassign the same workload definition—e.g., the same function in a FaaS model—to a previously used worker enclave. If the initial execution compromises the worker, due to a vulnerable workload or malicious input, any subsequent execution would be insecure. To uniquely identify each execution session—even when the workload definition remains the same—a randomized workload ID is required.

The workload ID is derived from a random nonce generated by the attester. One might wonder whether the client could generate this randomness instead. However, this only works when the workload involves a single client. In multi-client settings—i.e., Multi-Party Computation (MPC), all clients would need to agree on the nonce. To avoid this coordination complexity, we let the attester generate the random nonce.

Additionally, the workload ID is bound to the client’s public key. This allows the worker to authenticate the client when performing a Diffie-Hellman (DH) key exchange. It is essential to prevent unauthorized clients from provisioning runtime inputs to any given workload.

Client Attestation. The client verifies the worker’s authenticity via the attester. Specifically, the attester generates a quote with a hash of the worker’s information attached to the `REPORT_DATA`. The worker’s information includes the worker’s public key and the assigned workload ID. Additionally, the quote is bound to a client-generated nonce, ensuring its freshness; note that this nonce is different from that of workload ID. The client verifies the quote and, using the certified worker’s public key, establishes a shared key with the worker.

Workload Execution. Executing the workload within the worker enclave requires explicit authorization from the attester. Specifically, before starting execution, the worker must receive a MAC over the workload ID, generated by the attester using the shared key (established during the bootstrap phase). This MAC-signed workload ID serves as a cryptographic authorization signal, ensuring that the workload execution is triggered and tracked by the binding attester. Additionally, upon generating the MAC, the attester removes the worker from its fresh worker pool—i.e., it deletes the worker ID and all associated data, including the public key. This ensures that the worker is permanently marked as used and potentially *contaminated*, preventing it from being assigned any future workload. This guarantees that no worker enclave is ever reused.

Importantly, all clients involved in the workload must complete key exchange with the worker prior to the start of workload execution. This requirement stems from our design choice: once the workload begins, the worker is removed from the attester, and any subsequent client attestation request will be blocked. While the attester could, in principle, retain the worker until the workload completes, we avoid this option to simplify the protocol, as it offers little benefit to the client’s experience.

4.3 Security Analysis

We provide a security analysis of our protocol. The high-level structure of our analysis is as follows: First, we show that the only attack strategy that could possibly breach security is enclave reuse (Lemma 4.1). Specifically, we prove that as long as all worker enclaves are securely terminated upon completion of a workload,

the underlying confidential serverless computing remains secure. We then show that enclave reuse is impossible in our protocol (Lemma 4.2), thereby completing the argument. We elaborate on each step below.

LEMMA 4.1. *If worker enclaves are never reused, then the underlying confidential serverless computing is secure.*

PROOF. We show that the only possible attack strategy that could break the security of the underlying confidential serverless computing is the reuse of a compromised worker enclave. To formally define the adversary that will never launch enclave reuse attack, we assume that if a client completes Diffie-Hellman (DH) key exchange with a worker, then the worker is *fresh*—i.e., no workload execution has ever started before.

Suppose a client successfully establishes a shared key with a worker enclave. Then, the client must have verified the worker’s authenticity and integrity. More specifically, the client must have verified a quote that includes the expected `MR_ENCLAVE` of the attester. Thus, the attester enclave must behave as prescribed by the protocol, regardless of any potential misbehavior by a malicious gateway controller (e.g., deploying a fake attester). Similarly, during the mutual attestation step in the bootstrap phase, the attester must have received and verified the worker’s quote, which includes the expected `MR_ENCLAVE` of the worker, hard-coded in the attester enclave. Therefore, the worker enclave must behave as prescribed unless additional modules beyond the initial TCB—i.e., any module not measured in `MR_ENCLAVE`—are dynamically loaded.

However, since we assume that the worker enclave has not started any workload execution, the worker enclave must remain in its expected state—i.e., it follows the protocol. Thus, the shared key established between the worker and the client is never leaked to the attacker. Since the client encrypts the runtime input before sending it to the worker, and we have established that the worker behaves as prescribed up until workload execution starts, any runtime data beyond what is explicitly output by the client-defined workload remains inaccessible to the attacker. $\square$

LEMMA 4.2. *Worker enclaves are never reused.*

PROOF. We show that enclave reuse never occurs. More specifically, we prove that a client never completes a Diffie-Hellman (DH) key exchange with a previously used worker to send runtime data.

Suppose a client successfully completes DH key exchange. This implies that the client has verified an attester-generated quote that binds the worker’s public key used during the DH key exchange. The same quote also binds the workload ID, which is internally associated with a worker ID within the attester enclave.

Now, suppose for contradiction that any worker enclave with the worker ID bound to `CONFIG_ID` has started workload execution at least once before. Then, the worker must have previously received and verified an *attester-generated* MAC. This follows from the security of mutual attestation: The worker verifies the attester-generated quote using the expected `MR_ENCLAVE` of the attester, which is bound to `CONFIG_ID`. The attester also verifies the worker’s `CONFIG_ID` using its own `MR_ENCLAVE`. Thus, the worker must have established a shared MAC key with the genuine attester enclave.

We also note that while multiple worker enclaves may bind the same worker ID to their `CONFIG_ID` (e.g., due to malicious gateway

controller), only one of them can establish a shared secret key with the attester. Thus, a unique authorized worker enclave exists at any given time, meaning that if the workload execution has started, it is bound to a single enclave instance.

Recall that once the attester generates a MAC for a workload, it deletes the worker ID and all associated data, including the assigned workload ID. From this point on, the attester never generates another quote that binds to the same worker. Thus, the quote that the client received must have been issued before the attester generated the MAC for the worker. This implies that the workload ID bound to the client’s received quote must match the workload ID for which the MAC was generated.

However, recall that the workload ID is derived from an attester-generated nonce. Since the nonce is randomly generated for each workload request, even if the same workload definition is used, the resulting workload ID will always be different. Furthermore, the workload ID is also bound to the client’s public key, which is used during DH key exchange to establish a secure channel for transmitting runtime data, ultimately triggering workload execution. Thus, the workload ID is unique to the exact request session in which the client just completed the DH key exchange. This contradicts the assumption that workload execution had already started before the client’s key exchange. Therefore, worker enclaves are never reused. □

5 OUR PROTOTYPE FOR CONFIDENTIAL SERVERLESS COMPUTING

We now present our prototype system for confidential serverless computing, built on top of our nested attestation protocol presented in Section 4.

5.1 Gramine-SGX

Our prototype is built on the open-source Library OS, Gramine-SGX [40], which enables the execution of standard Linux binaries within Intel SGX enclaves. We first review the essential features of Gramine that are relevant to our prototype. Gramine provides a lightweight kernel compatibility layer that supports system calls, file operations, and inter-process communication, making it well-suited for executing ephemeral yet flexible confidential serverless workloads.

The initial enclave image in Gramine includes the library OS, along with the Platform Abstraction Layer (PAL), a thin abstraction layer that bridges interactions with the host OS. All system files and shared libraries from the host OS are dynamically loaded on demand. To ensure that these dynamically loaded modules are incorporated into the initial TCB—i.e., the boot-time measurement—Gramine provides an enclave *manifest*. The manifest contains precomputed hashes of all files that can be dynamically loaded—referred to as *trusted-files*. Before loading a trusted-file, its integrity is verified against the manifest-listed hash. Since the manifest is also embedded in the initial enclave image, it guarantees the integrity of all dynamically loadable modules.

Gramine also provides *encrypted-files* and *allowed-files*. Encrypted files securely store data within the Gramine instance, using encryption keys that remain protected inside the enclave. Allowed files,

```
{
  "function_id": "<function_hash>",
  "runtime_id": "python3.10",
  "inputs": {
    "input_x": "<client_x_pk_hash>",
    "input_y": "<client_y_pk_hash>"
  },
  "outputs": {
    "output_x": "<client_x_pk_hash>",
    "output_y": "<client_y_pk_hash>"
  },
  "handler": "handler.run"
}
```

Figure 4: Example workload definition.

```
def run():
    with open("/work/inputs/input_x/data.csv") as input_x_file:
        x = input_x_file.read()

    with open("/work/inputs/input_y/data.csv") as input_y_file:
        y = input_y_file.read()

    z = analyze(x, y) # do some analytics...

    with open("/work/outputs/output_x/result.txt", "w") as
    ↪ output_x_file:
        output_x_file.write(z)

    with open("/work/outputs/output_y/result.txt", "w") as
    ↪ output_y_file:
        output_y_file.write(z)
```

Figure 5: Example function (handler.py).

in contrast, are stored in plaintext without any integrity or confidentiality protection. They can be accessed or modified by the host without detection from Gramine, making them suitable as an interface for exchanging files between the enclave and the host system.

5.2 Workload Specification

Our prototype follows a Function-as-a-Service (FaaS)-like architecture and supports stateless workload execution. The workload logic, referred to as a *function*, is implemented as a standard Python package. The workload definition specifies the function entry point, referred to as the *handler*, which is the Python method invoked when workload execution starts. Additionally, our prototype supports multi-client workloads, where function inputs depend on runtime data from multiple clients—i.e., a Multi-Party Computation (MPC) setting. More specifically, the workload in our prototype is defined using a structured JSON object (e.g., Figure 4), which includes the following major information.

- **Function ID.** The cryptographic hash of the function package. When the function is provisioned to the worker enclave, its integrity is verified against this hash (c.f. step 3 in workload assignment in Figure 3).
- **Inputs.** A mapping of input identifiers to client identities, specifying which client provides which input. The client identity is represented by the hash of the public key used during key exchange—i.e., during client attestation.

- **Outputs.** A mapping specifying which client can access which output. The output is encrypted using the shared secret key established with the corresponding client.

It can also specify the runtime ID, which is the function runtime environment (e.g., language version).

An example function is shown in Figure 5. The encryption and decryption of inputs and outputs are transparently handled within the middleware layer in the worker enclave, abstracting these operations from the workload. Specifically, an encrypted input received from the client is decrypted using the shared secret key and placed in a dedicated input directory `/work/inputs/<input_id>`, where the function can access it as plaintext. Similarly, the function writes each output to a dedicated output directory `/work/outputs/<output_id>`, where it is encrypted before being sent back to the corresponding client. These directories are designated as *encrypted files* in the Gramine manifest, ensuring that plaintext data remains inaccessible to the host system.

5.3 Implementation Details

The function must be stored with integrity guarantees at runtime. Specifically, after the function is provisioned to the worker enclave and its integrity is verified against the specified function ID, it must be securely stored before being loaded. While trusted-files provide a mechanism for ensuring file integrity, they are statically bound to the initial TCB through the Gramine manifest. Since the function is part of the workload provisioned at runtime, trusted-files are not applicable. Another option is to use the in-memory file system—*tmpfs*—which stores files entirely within the enclave. However, this can lead to excessive memory usage, especially for large packages with multiple dependencies. Therefore, we choose to adopt encrypted-files. More specifically, the function is first provisioned to the worker enclave through allowed-files (during step 3 in Figure 3). After verifying its hash against the function ID, the function is stored as encrypted-files. The encryption key is generated and stored within the worker enclave, allowing each worker to securely offload the file while preserving its integrity. The destination directory is then added to the Python path, enabling dynamic function import at workload runtime.

We note that while we use encrypted-files, we do not rely on any persistent state (as mentioned in Section 2.4). The function stored as an encrypted-file is deleted once workload execution completes and the worker enclave is terminated.

We also note that while encrypted-files meet our integrity requirements, they provide stronger security than strictly necessary, as function confidentiality is not always required. An interesting future extension to Gramine would be support *dynamic trusted-files*, which would allow the trusted hash list to be updated at runtime. This would maintain integrity guarantees while avoiding encryption overhead.

5.4 Empirical Evaluation

We empirically evaluated our prototype system through experiments. All evaluations were conducted on Microsoft Azure using Standard_DC8ds_v3 instances, each equipped with 8 virtual CPUs,

Table 1: Latency Breakdown for XGBoost Training and Prediction Workloads (sec) with Varying EPC Sizes

Phase / Step	Training			Prediction		
	16G	8G	2G	16G	8G	2G
Bootstrap Phase						
Worker creation	52	23	10	54	23	10
Mutual attestation	~1	~1	~1	~1	~1	~1
Execution Phase						
Work assignment	10	11	11	10	10	9
Client attestation	~1	~1	~1	~1	~1	~1
Work execution						
Upload inputs	~1	~1	~1	~1	~1	~1
Function execution						
Depend. import	11	13	21	11	15	19
Data load & exec	~1	~1	~1	~1	~1	~1
Download outputs	~1	~1	~1	~1	~1	~1

64 GB of RAM, and 32 GB of EPC. We note that the Enclave Dynamic Memory Management (EDMM) [27] feature of Intel SGX was disabled in all of our experiments.

As a representative workload, we used machine learning training and prediction tasks for collaborative data analysis between two clients—a common industry use case for confidential serverless computing. Specifically, in the training workload, one client provides the training data, and the other receives the resulting trained model. In the prediction workload, one client provides the model while the other provides the input data; one or both clients receive the prediction results. For the ML algorithm, we used XGBoost [8], a widely adopted and moderately complex algorithm optimized for tabular data. We employed the official xgboost Python package. As the benchmark dataset, we used the standard Adult dataset (a.k.a., Census Income dataset) [19], retrieved from OpenML, which contains 32,561 training samples.

We measured the latency of each step in our protocol under varying EPC sizes, from 2 GB to 16 GB. Note that with a 1 GB EPC, the system failed to load the XGBoost module due to insufficient memory. Table 1 presents a detailed latency breakdown for both training and prediction workloads with 16 GB, 8GB, and 2 GB EPC configurations. We elaborate on the major parts of latency below.

Worker Creation. The most heavyweight operation is the worker creation step, whose overhead increases significantly with larger EPC sizes. This overhead primarily stems from the initial enclave measurement process: since each EPC page must be hash-extended, the time cost scales linearly with the allocated EPC size.

As we have seen in Section 4.1, our protocol can offload the bootstrap phase. We observe that this not only leads to a substantial reduction in latency but also enables the service to scale the EPC size without incurring additional overhead. For example, when examining the latency of dependency import, we find that it actually decreases as EPC scales; smaller EPC configurations trigger more frequent EPC paging. Thus, from the client’s perspective, the end-to-end latency improves with larger EPC sizes. This highlights an additional benefit of offloading the bootstrap phase—particularly for memory-intensive workloads.

A *tpm-enclave* provides an enclave-binding TPM for any target application enclave (henceforth called *app-enclave*). The *tpm-enclave*, upon initialized, generates a public-private key pair.

- **Initialization.** When creating an *app-enclave*, the *tpm-enclave* generates a random instance ID for the *app-enclave*. Then, the *app-enclave* is initialized with the `CONFIG_ID` set to the hash of: 1) the instance ID, and 2) the public key of the *tpm-enclave*.
- **Mutual Attestation.** The *app-enclave* and the *tpm-enclave* verify each other as follows:
 - (1) The *tpm-enclave* generates a quote with `REPORT_DATA` set to the hash of its public key. The *app-enclave* verifies the quote by checking that `MR_ENCLAVE` matches the expected measurement of the *tpm-enclave*, hard-coded in the *app-enclave*.
 - (2) The *app-enclave* also generates a quote with `REPORT_DATA` set to the hash of its own public key. The *tpm-enclave* verifies the quote by checking that `CONFIG_ID` matches the hash of: 1) its own instance ID, and 2) the TPM public key.
 The *tpm-enclave* and the *app-enclave* establish a shared key via Diffie-Hellman (DH) key exchange.
- **TPM Operation.** Any TPM operation (denoted *tpm-op*) requested from the *app-enclave* is invoked as follows:
 - (1) The *app-enclave* sends a MAC-signed TPM operation *tpm-op* request to the *tpm-enclave*.
 - (2) The *tpm-enclave* verifies the MAC and invokes *tpm-op*. Once the operation completes, it sends a MAC-signed completion message.
 - (3) The *app-enclave* verifies the MAC and proceeds with subsequent operations.
- **TPM Proof.** A remote client verifiably retrieves a tamper-proof report from the TPM as follows:
 - (1) The *tpm-enclave* generates a signed report that includes the requested TPM-stored value using its private key.
 - (2) The client receives and verifies the quote from the *app-enclave* by checking that:
 - `MR_ENCLAVE` matches the expected measurement of the *app-enclave*.
 - `CONFIG_ID` matches the hash of: 1) the instance ID, and 2) the TPM public key.
 - (3) The client then verifies the signed report from the TPM using the TPM public key.

Figure 6: Our protocol for realizing enclave-binding TPM on Intel SGX.

Workload Assignment. Another major overhead arises during the workload assignment step. This latency is primarily due to function integrity verification performed by the worker. Specifically, during this step, the function body is provisioned to the worker, which then computes its hash to verify integrity against the provided function ID. Since all required dependencies (e.g., XGBoost and pandas) are included as part of the function body, the cost of this step increases with the number and size of dependencies.

To mitigate this bottleneck, two potential approaches may be considered. The first approach, which can be immediately applied, is to include major dependencies (e.g., pandas, NumPy) in the initial TCB (i.e., via the Gramine manifest), thereby minimizing the size of the dynamically provisioned function. However, this increases the TCB and is less suitable for specialized or minor dependencies such as XGBoost. A longer-term solution is to extend Gramine to support dynamic trusted files, as discussed earlier, which would allow post-boot provisioning of integrity-protected files without bloating the initial TCB.

Dependency Import. Finally, we observe that importing dependencies dominates the function execution time. This includes libraries such as XGBoost and pandas (used for processing the CSV dataset). In our prototype, as described earlier, the function body is stored and loaded from encrypted-files. Thus, importing dependencies entails decrypting all associated files before use. We note, however, that this overhead does not significantly decrease even when using trusted-files. This is because dynamically loading files—regardless of the mechanism—requires integrity verification at some point. With large dependencies, this overhead is essentially inherent in any confidential serverless computing. Therefore, we believe the desirable effort is rather to minimize the function footprint itself. For instance, many Python packages (including XGBoost) bundle large shared libraries, most of whose functionalities are unused in

a given workload. Optimizing the structure of such dependencies to reduce the size of the provisioned function body is an interesting direction for future work.

6 IMPLEMENTING ENCLAVE-BINDING TPM

In the previous sections, we presented protocols for securing confidential serverless computing and its nested attestation. However, we believe that our KSS-based technique has broader implications beyond serverless computing. As a concrete example, we demonstrate how our technique can be leveraged to implement an *enclave-binding* TPM on Intel SGX.

Enclave-binding TPM. The Trusted Platform Module (TPM) is a widely used hardware security module that provides trusted services such as cryptographic operations, secure key storage, and attestation. While TPMs have traditionally been implemented as discrete hardware chips at the platform level, per-VM or per-process TPMs are becoming increasingly important in the context of confidential computing. For example, confidential VMs often rely on virtual TPMs (vTPMs) [29, 33] to support measured boot and manage attestation keys. Furthermore, Intel TDX, a TEE for VM isolation, introduces Runtime Measurement Registers (RTMRs), which enable runtime measurement of dynamic modules inside individual VMs or enclaves—akin to TPM PCR extensions.

When implementing such a per-enclave TPM on Intel SGX, a key challenge is to ensure that the target application enclave is exclusively bound to a single TPM enclave. More specifically, the application enclave must not be able to re-establish a connection with a different TPM enclave, even post-compromise, ensuring that the TPM state bound to it cannot be reverted. This is particularly important for realizing RTMR-like functionality: if the RTMR state can be rolled back or reset, it no longer reflects the genuine runtime state of the application enclave.

We observe that our KSS-based technique essentially provides a mechanism to securely bind enclaves to one another. This capability enables the implementation of an enclave-binding TPM on Intel SGX, supporting enclave-local trusted functionalities, such as RTMRs.

6.1 Our Protocol

Our protocol is summarized in Figure 6, and we elaborate on the key techniques below.

Binding enclave to TPM. As mentioned, the target application enclave (henceforth referred to as *app-enclave*) must be exclusively bound to a single TPM enclave so that its TPM state will not be reverted regardless of the underlying security assumption of the app-enclave. To this end, we utilize a random *instance ID* generated by the tpm-enclave to identify app-enclave. The tpm-enclave generates a random instance ID to track the app-enclave and binds it to the app-enclave’s CONFIG_ID. During attestation, the tpm-enclave verifies that the app-enclave’s CONFIG_ID matches its assigned instance ID before establishing a shared key. This ensures that the app-enclave cannot re-establish a connection with a different attester, guaranteeing one-time TPM provisioning.

Enforcing Completion of TPM Operations. Emulating TPM operations requires not only enabling the operations but also enforcing their completion before subsequent execution. For example, consider a *PCR Extension* operation invoked within the app-enclave for measuring a dynamically loaded module. To maintain security, the PCR Extension must be completed before the module is loaded. Simply enabling the operation alone is insufficient—enforcing execution order is necessary.

Recall that our nested attestation protocol employs MAC-based authorization from the attester to start workload execution within a worker enclave. This effectively acts as a cryptographic acknowledgment (ack) that confirms the completion of a specific operation. Our protocol adopts the same approach for TPM operations. Specifically, the application enclave can proceed with subsequent operations (e.g., module loading) only after receiving a MAC-signed completion message from the tpm-enclave, confirming the successful execution of the requested TPM operation.

Tamper-proof TPM Report. A crucial aspect of TPM functionality is the ability to provide verifiable, tamper-proof reports to remote clients. To achieve this, the public key of the tpm-enclave is bound to the app-enclave’s CONFIG_ID and is used to generate signed reports within the TPM. This guarantees that even if the app-enclave is compromised, the remote client can still retrieve authentic TPM values and verify their integrity.

7 RELATED WORKS

Finally, we compare our work with related prior works.

Offloading Cold Start in Serverless Computing. Serverless computing systems typically employ cold start offloading strategies to reduce the latency of online workload execution. We observe that the literature predominantly explores two orthogonal strategies.

The first strategy is *worker reuse with sanitization* [43]. Specifically, a worker instance is retained in a “warm” pool after use.

Before being reassigned, the worker is sanitized to remove any residual state. This enables fast responses to successive requests without paying the cost of full container or enclave bootstrapping for all but the first invocation.

An alternative strategy is to *decouple the workload from the initial worker instance*, minimizing the cost of workload initialization [20, 23, 25]. Specifically, workers are pre-initialized with platform components—such as the language runtime and system middleware—while the actual workload is kept as a placeholder. This allows the system to pre-boot workers in advance and quickly process any workload dynamically provisioned at runtime.

Our work aligns with the latter strategy. We explicitly avoid reusing workers, opting instead to dynamically provision workloads into pre-booted worker enclaves, offloading enclave bootstrapping. To securely enforce this reuse-free model in confidential computing, we introduce a KSS-based nested attestation protocol that prevents post-compromise enclave reuse.

Intra-enclave Isolation. We observe that the first cold start offloading strategy—i.e., worker reuse with sanitization—has also been explored in the context of confidential computing. The current state-of-the-art is the recent elegant work by Zhao et al. [44]. Briefly, their approach is to snapshot a clean worker enclave and efficiently rewind the enclave state to the snapshot before each reuse. Crucially, enclave rewinding must be performed securely by a trusted module that is isolated from the workload itself. In other words, this requires *intra-enclave isolation* to sandbox workload execution. This requirement also arises if we attempt to realize a reuse-free model without our KSS-based technique: a trusted module, isolated from the workload, would be needed to block any subsequently assigned workloads. However, as several prior efforts have shown, implementing intra-enclave isolation in Intel SGX is inherently complex and hard. Due to the lack of built-in features such as privilege separation, such isolation typically relies on Software Fault Isolation (SFI).

To briefly review prior efforts in implementing SFI on Intel SGX, two general approaches have been explored: relying on specialized hardware extensions, or using purely software-based techniques. For the former, a common approach [35, 37, 45] has been to leverage Intel Memory Protection Extensions (MPX) [31, 34], an architectural extension that enforces bounds checking to restrict memory access. However, Intel MPX has since been deprecated and is no longer supported. Purely software-based solutions also exist [36, 44], but they typically require complex instrumentation and static analysis, and often incur performance and implementation overhead. In either case, SFI requires a specialized compiler to generate SFI-instrumented binaries. This requirement often becomes a bottleneck in real-world, industry use cases where existing binary code, such as shared libraries, is integrated into the workload. These practical limitations have motivated our design choice to avoid intra-enclave isolation.

Runtime Measurement. One straightforward approach to realizing the reuse-free model is to utilize a hardware feature that supports runtime measurement for each enclave. Specifically, a dynamically provisioned workload can be measured at runtime via

hash extension, and the resulting digest is stored in a hardware-protected register, serving as a tamper-proof record of use. For example, the Runtime Measurement Registers (RTMRs) in Intel TDX [9] provide such functionality, allowing a virtual machine to measure dynamically loaded modules during execution. One of the key insights of our work is that RTMR-like functionality can be approximated using only an immutable ID. This technique could be applied to other TEE technologies that lack built-in runtime measurement support, which is an interesting future work.

8 CONCLUSION

In this work, we have presented a novel nested attestation scheme for confidential serverless computing on Intel SGX, leveraging KSS as a post-compromise identifier. We have also presented how our technique can be applied to implement an enclave-binding TPM on Intel SGX. A promising future direction, with broader impact on general confidential computing using Intel SGX, is to implement an Intel SGX counterpart to Intel TDX's RTMR.

ACKNOWLEDGMENTS

We would like to thank Yuya Kato, Takuya Hayashi, Mitsuaki Makino, and Raiki Tanaka for valuable discussions related to this work. We are also grateful to the engineering team at Acompany Co., Ltd. for their assistance with our implementation.

REFERENCES

- [1] Adil Ahmad, Juhee Kim, Jaebaek Seo, Insik Shin, Pedro Fonseca, and Byoungyoung Lee. 2021. CHANCEL: Efficient Multi-client Isolation Under Adversarial Programs. In *NDSS*, Vol. 6. 221–230.
- [2] Fritz Alder, N Asokan, Arseny Kurnikov, Andrew Paverd, and Michael Steiner. 2019. S-faas: Trustworthy and accountable function-as-a-service using intel sgx. In *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop*. 185–199.
- [3] AMD. April 23, 2025. AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. <https://www.amd.com/system/files/TechDocs/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf>.
- [4] Ferdinand Brasser, Urs Müller, Alexandra Dmitrienko, Kari Kostiaainen, Srdjan Capkun, and Ahmad-Reza Sadeghi. 2017. Software grand exposure: {SGX} cache attacks are practical. In *11th USENIX workshop on offensive technologies (WOOT 17)*.
- [5] David Brumley and Dan Boneh. 2005. Remote timing attacks are practical. *Computer Networks* 48, 5 (2005), 701–716.
- [6] Guoxing Chen and Yinqian Zhang. 2022. {MAGE}: Mutual attestation for a group of enclaves without trusted third parties. In *31st USENIX Security Symposium (USENIX Security 22)*. 4095–4110.
- [7] Hongbo Chen, Haobin Hiroki Chen, Mingshen Sun, Kang Li, Zhaofeng Chen, and XiaoFeng Wang. 2023. A verified confidential computing as a service framework for privacy preservation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4733–4750.
- [8] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [9] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. 2024. Intel tdx demystified: A top-down approach. *Comput. Surveys* 56, 9 (2024), 1–33.
- [10] Graeme Connell. 2022. Technology Deep Dive: Building a Faster ORAM Layer for Enclaves. <https://signal.org/blog/building-faster-oram/>.
- [11] Victor Costan and Srinivas Devadas. 2016. Intel SGX explained. *Cryptology ePrint Archive* (2016).
- [12] Ivan Damgård and Claudio Orlandi. 2010. Multiparty computation for dishonest majority: From passive to active security at low cost. In *Annual cryptography conference*. Springer, 558–576.
- [13] Daniel Escudero, Vipul Goyal, Antigoni Polychroniadou, Yifan Song, and Chenkai Weng. 2023. Superpack: Dishonest majority mpc with constant online communication. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 220–250.
- [14] Oded Goldreich and Rafail Ostrovsky. 1996. Software protection and simulation on oblivious RAMs. *Journal of the ACM (JACM)* 43, 3 (1996), 431–473.
- [15] Jinyu Gu, Bojun Zhu, Mingyu Li, Wentai Li, Yubin Xia, and Haibo Chen. 2022. A {Hardware-Software} co-design for efficient {Intra-Enclave} isolation. In *31st USENIX Security Symposium (USENIX Security 22)*. 3129–3145.
- [16] Tianlin Huo, Xiaoni Meng, Wenhao Wang, Chunliang Hao, Pei Zhao, Jian Zhai, and Mingshu Li. 2020. Bluethunder: A 2-level directional predictor based side-channel attack against sgx. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2020), 321–347.
- [17] Intel. April 23, 2025. Intel® Trust Domain Extensions (Intel® TDX). <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html>.
- [18] Thomas Knauth, Michael Steiner, Somnath Chakrabarti, Li Lei, Cedric Xing, and Mona Vij. 2018. Integrating remote attestation with transport layer security. *arXiv preprint arXiv:1801.05863* (2018).
- [19] Ron Kohavi et al. 1996. Scaling up the accuracy of naive-bayes classifiers: A decision-tree hybrid. In *Kdd*, Vol. 96. 202–207.
- [20] Anisha Kumari and Bibhudatta Sahoo. 2024. ACPM: adaptive container provisioning model to mitigate serverless cold-start. *Cluster Computing* 27, 2 (2024), 1333–1360.
- [21] Enrique Larraia, Emmanuela Orsini, and Nigel P Smart. 2014. Dishonest majority multi-party computation for binary circuits. In *Annual Cryptology Conference*. Springer, 495–512.
- [22] Mingyu Li, Yubin Xia, and Haibo Chen. 2021. Confidential serverless made efficient with plug-in enclaves. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 306–318.
- [23] Rui Li, Devesh Tiwari, and Gene Cooperman. 2024. WarmSwap: Sharing Dependencies for Accelerating Cold Starts in Serverless Functions. *arXiv preprint arXiv:2409.09202* (2024).
- [24] Xupeng Li, Xuheng Li, Christoffer Dall, Ronghui Gu, Jason Nieh, Yousuf Sait, and Gareth Stockwell. 2022. Design and Verification of the Arm Confidential Compute Architecture. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [25] Zijun Li, Quan Chen, and Minyi Guo. 2021. Pagurus: Eliminating cold startup in serverless computing with inter-action container sharing. *arXiv preprint arXiv:2108.11240* (2021).
- [26] Sinisa Matetic, Mansoor Ahmed, Kari Kostiaainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels, and Srdjan Capkun. 2017. {ROTE}: Rollback protection for trusted execution. In *26th USENIX Security Symposium (USENIX Security 17)*. 1289–1306.
- [27] Frank McKeen, Ilya Alexandrovich, Ittai Anati, Dror Caspi, Simon Johnson, Rebekah Leslie-Hurd, and Carlos Rozas. 2016. Intel® software guard extensions (intel® sgx) support for dynamic memory management inside an enclave. In *Proceedings of the Hardware and Architectural Support for Security and Privacy 2016*. 1–9.
- [28] Frank McKeen, Ilya Alexandrovich, Alex Berenzon, Carlos V Rozas, Hisham Shafi, Vedvyas Shanbhogue, and Uday R Savagaonkar. 2013. Innovative instructions and software model for isolated execution. *Hasp@ isca* 10, 1 (2013).
- [29] Vikram Narayanan, Claudio Carvalho, Angelo Ruocco, Gheorghe Almasi, James Bottomley, Mengmei Ye, Tobin Feldman-Fitzthum, Daniele Buono, Hubertus Franke, and Anton Burtsev. 2023. Remote attestation of confidential VMs using ephemeral vTPMs. In *Proceedings of the 39th Annual Computer Security Applications Conference*. 732–743.
- [30] Intel Newsroom. 2021. Intel SGX Protects German Electronic Patient Records. <https://download.intel.com/newsroom/archive/2025/en-us-2021-04-27-intel-sgx-protects-german-electronic-patient-records.pdf>.
- [31] Oleksii Oleksenko, Dmitrii Kuvaishii, Pramod Bhatotia, Pascal Felber, and Christof Fetzer. 2018. Intel mpx explained: A cross-layer analysis of the intel mpx system stack. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 2, 2 (2018), 1–30.
- [32] Joongun Park, Seunghyo Kang, Sanghyeon Lee, Taehoon Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2024. Hardware-hardened sandbox enclaves for trusted serverless computing. *ACM Transactions on Architecture and Code Optimization* 21, 1 (2024), 1–25.
- [33] Ronald Perez, Reiner Sailer, Leendert van Doorn, et al. 2006. vTPM: virtualizing the trusted platform module. In *Proc. 15th Conf. on USENIX Security Symposium*. 305–320.
- [34] Ramu Ramakesavan, Dan Zimmerman, and Pavithra Singaravelu. 2015. Intel memory protection extensions (intel mpx) enabling guide. *no. April* (2015).
- [35] Vasily A Sartakov, Daniel O’Keeffe, David Eysers, Lluís Vilanova, and Peter Pietzuch. 2021. Spons & Shields: practical isolation for trusted execution. In *Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 186–200.
- [36] Jaebaek Seo, Byoungyoung Lee, Seong Min Kim, Ming-Wei Shih, Insik Shin, Dongsu Han, and Taesoo Kim. 2017. SGX-shield: Enabling address space layout randomization for SGX programs. In *NDSS*.

- [37] Youren Shen, Hongliang Tian, Yu Chen, Kang Chen, Runji Wang, Yi Xu, Yubin Xia, and Shoumeng Yan. 2020. Occlum: Secure and efficient multitasking inside a single enclave of intel sgx. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 955–970.
- [38] Emil Stefanov, Marten van Dijk, Elaine Shi, T-H Hubert Chan, Christopher Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. 2018. Path ORAM: an extremely simple oblivious RAM protocol. *Journal of the ACM (JACM)* 65, 4 (2018), 1–26.
- [39] Afonso Tinoco, Sixiang Gao, and Elaine Shi. 2023. {EnigMap}:{External-Memory} Oblivious Map for Secure Enclaves. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4033–4050.
- [40] Chia-Che Tsai, Donald E Porter, and Mona Vij. 2017. Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. 645–658.
- [41] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A Gunter. 2017. Leaky cauldron on the dark land: Understanding memory side-channel hazards in SGX. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2421–2434.
- [42] Conrad Watt, John Renner, Natalie Popescu, Sunjay Cauligi, and Deian Stefan. 2019. Ct-wasm: type-driven secure cryptography for the web ecosystem. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 1–29.
- [43] Tianyi Yu, Qingyuan Liu, Dong Du, Yubin Xia, Binyu Zang, Ziqian Lu, Pingchao Yang, Chenggang Qin, and Haibo Chen. 2020. Characterizing serverless platforms with serverlessbench. In *Proceedings of the 11th ACM Symposium on Cloud Computing*. 30–44.
- [44] Shixuan Zhao, Pinshen Xu, Guoxing Chen, Mengya Zhang, Yinqian Zhang, and Zhiqiang Lin. 2023. Reusable enclaves for confidential serverless computing. In *32nd USENIX security symposium (USENIX Security 23)*. 4015–4032.
- [45] Wenjia Zhao, Kangjie Lu, Yong Qi, and Saiyu Qi. 2020. Mptee: bringing flexible and efficient memory protection to intel sgx. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–15.